π-Former: Succinct Proofs of Transformer Inference via a SNARK-Friendly Architecture

Hideaki Takahashi
Columbia University
ht2673@columbia.edu

Abstract

Verifiable transformer inference lets an untrusted operator prove that a posted output was produced by a committed model, without revealing the weights. The obstacle is that a transformer block is built from exactly the operations that finite-field SNARKs find expensive: softmax, GeLU, LayerNorm, and dense projections. Prior systems leave the model untouched and pay this cost on every block.

We present π-Former, a succinct argument whose central design choice is to co-design the *model* so the proof system has nothing expensive to absorb. Three architectural changes drive the design: (i) softmax attention is replaced by a kernel attention whose feature map is realised as an additively decomposed learnable lookup, so the only non-arithmetic operation is a structured Lasso table look-up; (ii) every projection is constrained to ternary entries scaled by a per-layer scalar, eliminating all field multiplications from weight contractions; and (iii) LayerNorm is reformulated as an integer protocol provable through sumcheck and chunked range proofs. We build a transparent BN254 SNARK on top with Hyrax commitments, batching every per-block sub-protocol into a constant number of model-level sumchecks, a globally shared range-multiplicity commitment, and a zerocheck-folded inverse polynomial that removes $B{\cdot}C$ Hyrax commitments per range bucket. Residual connections require no proof: they are verified by linearity of the commitment scheme. We prove end-to-end soundness under discrete log on BN254 G1 and implement a complete prototype in ∼19k lines of Rust with a bit-identical PyTorch training pipeline.

1 Introduction

Large language models are increasingly deployed as black-box services. The user submits a prompt, an opaque operator runs a proprietary model, and the user has no cryptographic guarantee that the returned output came from the advertised model. Verifiable inference [1, 10, 12, 15] closes this gap with a succinct non-interactive argument: the operator commits to the weights once, and for every input produces a small proof that the verifier checks without re-executing the model and without learning the weights.

The core difficulty is that a transformer block is built from operations that finite-field SNARKs find expensive. Softmax requires exponentials and a row-wise division; GeLU is transcendental; LayerNorm contains a square root and a division by a running variance; a dense projection costs one field multiplication per weight entry. Prior work attacks this on the proof-system side: zkLLM [15] introduces `tlookup`, a parallelised lookup argument tailored to softmax and SwiGLU; zkGPT [12] pairs Plonky2 with constraint fusion to drive down the cost of approximating softmax. Both keep the model fixed and pay an "unfriendly-operation tax" on every block.

Our approach. π-Former starts from a different premise: change the *model* so that the proof system has nothing expensive to absorb. Three architectural principles drive the design:

- **SNARK-friendly operators.** Replace every operation without a compact arithmetic representation. Softmax becomes a kernel attention whose only non-arithmetic component is a structured table lookup; LayerNorm becomes an integer protocol provable by sumcheck plus chunked range proofs.
- **Ternary projections for free contractions.** Constrain every projection matrix to entries $\{-1, 0, +1\}$ scaled by a per-layer scalar α [11]. A degree-2 sumcheck against a ternary weight folds the equality polynomial against ternary entries with $+=$/$-=$/skip; no field multiplications appear in the contraction.
- **Learnable lookup tables.** The element-wise non-linearity ϕ is itself learnable: it is parametrised as a sum of small sub-tables and trained end-to-end. Under Lasso [14], prover cost depends only on table size and query count, not on table contents, so training adapts table values at zero proving cost.

Together these reduce a transformer block to operations that map directly to a low-degree sumcheck or to a Lasso lookup over a small structured table.

Model-level batching. On top of this architecture we build a proof system around a single observation: the same protocol runs once per block with independent inputs but identical structure. π-Former runs six model-level sumchecks (one per protocol type) that fold all L per-block claims via a Fiat–Shamir random linear combination; the inner sumcheck transcript is shared across blocks. The same idea applies to Lasso (one global multi-instance proof per lookup family), to range checks (one shared multiplicity commitment per width bucket spanning the entire model), to residuals (homomorphic Hyrax addition; no proof at all), and to the final opening stage (deferred batch accumulators finalised in one MSM pair per parameter group). For the range proof specifically, we replace the explicit Hyrax commitment to each LogUp inverse polynomial by a zerocheck folded into the bucket sumcheck; this removes $B \cdot C$ commitments per bucket without changing the per-round prover cost.

Contributions.

1. A SNARK-friendly transformer block (§3) in which attention, normalisation, and projection each map to a sumcheck or a Lasso lookup, combining learnable lookup attention, ternary projections, and a constraint-fused integer LayerNorm.
2. A complete protocol (§4) batching all L per-block subproofs into six model-level cross-block sumchecks, one global Lasso for the ϕ family, and a bucket-batched range proof shared across the whole model.
3. A zerocheck-folded LogUp range proof (§3.5, Theorem 5) that eliminates $B \cdot C$ Hyrax commitments per width bucket while keeping the bucket sumcheck cubic and soundness within $O(n/|\mathbb{F}|)$.
4. End-to-end soundness (§5, Theorem 8) reducing to discrete log on BN254 G1, and a working prototype in ∼19k Rust LoC with a bit-identical PyTorch training pipeline (§6).

Threat model. We target the standard non-interactive argument-of-knowledge setting in the random-oracle model. The verifier holds a verifying key vk binding the weights and lookup tables, a public input $\mathbf{x}_{\text{in}}$, and a claimed output $\mathbf{y}$, and accepts iff $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) = \mathbf{y}$ for the committed model. Soundness is computational under discrete log on BN254 G1, the standard assumption for the Hyrax PCS [18]. The current implementation is not zero-knowledge: weight commitments are public and intermediate sumcheck evaluations are revealed. The standard masking extension is discussed in §7.

2 Preliminaries

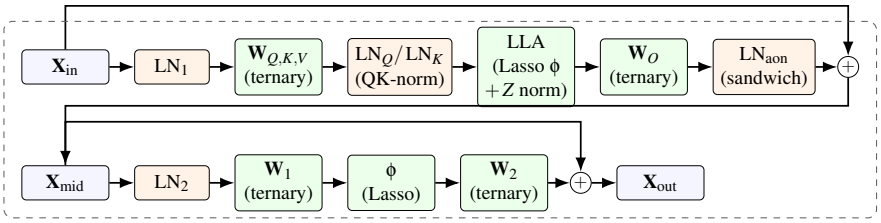
Notation. We work over the BN254 scalar field $\mathbb{F}$ ($r \approx 2^{254}$) and the BN254 G1 group $\mathbb{G}$. We use L for the number of blocks, T for the sequence length, d for the embedding dimension, and d_{ff} for the FFN width; the prototype is single-head ($d_h = d$). A multilinear polynomial $\widetilde{f}$ on n variables is identified with its 2^n evaluations on $\{0,1\}^n$, and the equality polynomial $\widetilde{\text{eq}}(r,x) = \prod_i (r_i x_i + (1-r_i)(1-x_i))$ satisfies $\sum_x \widetilde{\text{eq}}(r,x)\widetilde{f}(x) = \widetilde{f}(r)$. Real-valued tensors are quantised to integers and embedded in $\mathbb{F}$ before commitment; our training pipeline runs the same integer arithmetic, so the exported witness matches the field circuit bit-for-bit.

Transformer block. A pre-norm block [17] computes $\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \text{Att}(\text{LN}(\mathbf{X}_{\text{in}}))$ and $\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \text{FFN}(\text{LN}(\mathbf{X}_{\text{mid}}))$, with softmax attention $\text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V}$ and LayerNorm [3] $\gamma \odot (\mathbf{x}-\mu)/\sqrt{\sigma^2+\varepsilon} + \beta$. The non-arithmetic primitives (exp, division, square root) dominate SNARK complexity in prior work.

Sumcheck. The sumcheck protocol [13,16] reduces a claim $H = \sum_{x \in \{0,1\}^n} f(x)$ for an MLE f of individual degree δ to a single evaluation $f(r)$ at random r, with soundness error $\leq n\delta/|\mathbb{F}|$ and verifier cost $O(n\delta)$. π-Former uses the degree-2 variant for products of two MLEs, the cubic variant for LayerNorm variance and our zerocheck-folded range proof, and a multi-batched variant $\sum_k \lambda_k \sum_x f_k(x) g_k(x)$ for cross-block batching.

Hyrax PCS. Hyrax [18] arranges $2^n = 2^{\nu+\sigma}$ MLE evaluations as a $2^\nu \times 2^\sigma$ matrix and commits to each row via a vector Pedersen commitment over transparent generators. The scheme is computationally binding under DL on $\mathbb{G}$, requires no trusted setup, and gives $O(\sqrt{N})$ commitments, openings, and proof size. Crucially, the row-Pedersen structure is linear: $\mathsf{Com}(\mathbf{A}) + \mathsf{Com}(\mathbf{B}) = \mathsf{Com}(\mathbf{A}+\mathbf{B})$, which π-Former exploits to verify residual connections without any proof.

Lasso. Lasso [2,5,14] proves that N queries are consistent with a public sub-table $T \in \mathbb{F}^{2^m}$ via one batched sumcheck. A composite table that decomposes *additively* into c sub-tables (Lasso's structured condition) is committed as c small tables of size $2^{B/c}$. Lasso's defining property for our design is that prover and verifier cost depend on table size and query count, but are independent of table *contents*: training the entries of T has zero proving-cost impact.



Cross-block batching. The five LNs per block (LN_1, QK-norm LN_Q/LN_K, sandwich LN_{aon}, LN_2), plus the final LN, are folded into one *model-level* batched LN proof. Ternary projections fold into `qkv`/`oproj`/`ffn` sumchecks; LLA inner products fold into `batch_attn_out`/`ctx`; the row-normaliser Z is enforced by a cubic multi-batched sumcheck with range proofs on the floor-division residuals (§3.1); Lasso ϕ folds into one global multi-instance proof per lookup family; range checks fold into one shared multiplicity commitment per width bucket; residuals are homomorphic Hyrax addition.

Figure 1: A single π-Former block. Boxes are matrices; rounded green nodes are operations the SNARK proves; orange nodes are LayerNorms. *Sandwich norm* (LN_{aon}) is applied to the attention output before the residual; *QK-norm* (LN_Q, LN_K) is applied to $\mathbf{Q},\mathbf{K}$ after the ternary projection and before ϕ. Ternary projections contain no field multiplications in the contraction; Lasso lookup cost is independent of table contents; residual $+$ is checked by homomorphic addition of Hyrax commitments and requires no proof.

3 The π-Former Architecture

A π-Former block (Figure 1) is a sandwich-normalised linear-attention block. With $\mathbf{X}_{\text{in}} \in \mathbb{F}^{T\times d}$, the block computes

$$
\begin{aligned}
\mathbf{X}_{\text{n1}} &= \text{LN}_1(\mathbf{X}_{\text{in}}),\\
\mathbf{Q},\mathbf{K},\mathbf{V} &= \mathbf{X}_{\text{n1}}\mathbf{W}_{Q,K,V} \ \text{(ternary)},\\
\mathbf{Q}^{\text{n}},\mathbf{K}^{\text{n}} &= \text{LN}_Q(\mathbf{Q}),\ \text{LN}_K(\mathbf{K}),\\
\mathbf{N} &= \phi(\mathbf{Q}^{\text{n}})\big(\phi(\mathbf{K}^{\text{n}})^\top\mathbf{V}\big),\\
Z_t &= \phi(\mathbf{q}^{\text{n}}_t)^\top\textstyle\sum_s\phi(\mathbf{k}^{\text{n}}_s),\\
\mathbf{Y}^{\text{att}}_{tj} &= \lfloor S\cdot\mathbf{N}_{tj}/Z_t\rfloor,\\
\mathbf{X}_{\text{mid}} &= \mathbf{X}_{\text{in}} + \text{LN}_{\text{aon}}(\mathbf{Y}^{\text{att}}\mathbf{W}_O + \mathbf{b}_O),\\
\mathbf{X}_{\text{out}} &= \mathbf{X}_{\text{mid}} + \phi(\text{LN}_2(\mathbf{X}_{\text{mid}})\mathbf{W}_1)\mathbf{W}_2.
\end{aligned}
$$

After L such blocks the model applies a final LayerNorm and a ternary LM head; the model contains $5L+1$ LayerNorms in total. The scale S re-introduces fractional precision before the integer floor division.

Two of the five LayerNorms per block are not in a textbook transformer but are essential for our integer pipeline. *QK-norm* (LN_Q,LN_K) pins the range of $\mathbf{Q},\mathbf{K}$ so that $\phi(\mathbf{Q}^{\text{n}}),\phi(\mathbf{K}^{\text{n}})$ and Z stay in the lookup-table range under any per-layer ternary scale. *Sandwich norm* (LN_{aon}) absorbs the unbounded magnitude of linear attention before the residual: softmax is row-stochastic and naturally bounded, but $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top\mathbf{V})$ is not, and an unnormalised residual destabilises downstream LayerNorms. Both share one LN sub-protocol (§3.4); their only cost is extra LN witnesses, folded into the model-level batched LN proof and the bucketed range batch.

3.1 Learnable Lookup Attention

Softmax attention requires T^2 exponentials and T row-wise divisions. We replace the kernel with a positive-definite inner product $\kappa(\mathbf{q},\mathbf{k}) = \langle\phi(\mathbf{q}),\phi(\mathbf{k})\rangle$ on the post-QK-norm tensors, where $\phi:\mathbb{R}^{d_h}\to\mathbb{R}^{d_h}$ is an element-wise structured lookup (§3.2). The block computes the numerator $\mathbf{N}_{tj} = \phi(\mathbf{q}^{\text{n}}_t)^\top(\sum_s\phi(\mathbf{k}^{\text{n}}_s)\mathbf{v}_s^\top)_j$ and the per-row denominator $Z_t = \phi(\mathbf{q}^{\text{n}}_t)^\top\sum_s\phi(\mathbf{k}^{\text{n}}_s)$. The context matrix $\mathbf{M}=\sum_s\phi(\mathbf{k}^{\text{n}}_s)\mathbf{v}_s^\top\in\mathbb{F}^{d_h\times d_h}$ is computed once per block, giving $O(Td_h^2)$ prover cost instead of $O(T^2d_h)$ [7].

Floor-division proof for row normalisation. π-Former proves $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S\mathbf{N}_{tj}/Z_t\rfloor$ in-circuit. The witness includes auxiliary tensors rem and diff with $\text{rem}_{tj} = S\mathbf{N}_{tj} - Z_t\mathbf{Y}^{\text{att}}_{tj}$ and $\text{diff}_{tj} = Z_t - 1 - \text{rem}_{tj}$, all committed via Hyrax. At a random $r_{\text{norm}}\in\mathbb{F}^{t_{\text{bits}}+d_{\text{bits}}}$ one cubic multi-batched sumcheck merges $S\mathbf{N}-\text{rem}-Z\mathbf{Y}^{\text{att}}=0$ and $\lambda(Z-1-\text{rem}-\text{diff})=0$ under a Fiat–Shamir scalar λ. Range proofs on rem, diff (in the 64-bit bucket of §3.5) enforce both to be non-negative, which combined with the second identity gives $0\le\text{rem}<Z$ — the unique floor-division witness. A second sumcheck binds Z_t to the committed $\phi(\mathbf{Q}^{\text{n}}),\phi(\mathbf{K}^{\text{n}})$; a causal variant binds Z to a prefix sum.

Binding lookups to post-norm tensors. The Lasso queries for ϕ are evaluated on $\mathbf{Q}^{\text{n}},\mathbf{K}^{\text{n}}$, not the pre-norm $\mathbf{Q},\mathbf{K}$. The prover commits both $C_{\mathbf{Q}},C_{\mathbf{K}}$ (used by the QKV sumcheck) and $C_{\mathbf{Q}^{\text{n}}},C_{\mathbf{K}^{\text{n}}}$ (used as ϕ inputs); the global ϕ Lasso binds its query indices to the post-norm commitments via one shared Hyrax batch open, and the LN sub-proof for LN_Q,LN_K ties pre- and post-norm together.

3.2 Additive sub-table feature map

Rather than fix a kernel, π-Former parametrises ϕ as a sum of learnable sub-table lookups. With input bit-width B, decomposition depth $c \mid B$, and chunk width $m = B/c$, define the integer index $x_{\text{int}} = \text{clamp}(\lfloor x/s \rfloor, 0, 2^B - 1)$ and chunk extractor $\chi_i(x_{\text{int}}) = (x_{\text{int}} \gg im) \,\&\, (2^m - 1)$. Then

$$\phi(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\text{int}})\big], \tag{1}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^m}$ are learnable sub-tables. They are initialised so $\sum_i T_i \approx \text{GELU}$ and trained end-to-end via the straight-through estimator [4]. For $B = 16, c = 2$ this replaces a 65,536-entry monolithic table with two 256-entry sub-tables, matching Lasso's structured-decomposability condition (§2). Because Lasso prover cost is independent of table values, training the entries of every T_i end-to-end imposes zero proving-cost penalty.

3.3 Ternary projections

Following BitNet b1.58 [11], every projection matrix ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$, and the LM head) is constrained to entries in $\mathcal{T} = \{-1, 0, +1\}$ scaled by a learnable per-layer scalar $\alpha \in \mathbb{F}$, so the forward pass is $\mathbf{y} = \alpha(\mathbf{x}\mathbf{W}) + \mathbf{b}$ with $\mathbf{W} \in \mathcal{T}^{m \times n}$. Training uses the magnitude-thresholded $w \mapsto \text{sign}(w) \cdot [|w| > 0.7\overline{|w|}]$ map with the straight-through estimator; α is multiplied in after the matrix product so the in-circuit weight stays ternary.

In the SNARK, the contraction $\mathbf{Y}(r_t, r_d) - \mathbf{b}(r_d) = \alpha \sum_k \mathbf{X}(r_t, k)\mathbf{W}(k, r_d)$ is a degree-2 sumcheck that materialises only the cheap vector $\alpha \cdot \mathbf{W}(\cdot, r_d)$. The prover folds the equality polynomial against ternary entries with $+=$/$-=$/skip; no field multiplication appears in the contraction.

3.4 Constraint-fused LayerNorm

Let $\mathbf{x} \in \mathbb{Z}^d$ be a row, $\text{sum\_x} = \sum_j x_j$, $\text{sq\_sum\_x} = \sum_j x_j^2$, and $\sigma = \lfloor \sqrt{\text{var\_x}}/d \rfloor$. The integer variance is $\text{var\_x} = d(d\,\text{sq\_sum\_x} - \text{sum\_x}^2)$, which expands $\sum_j (dx_j - \text{sum\_x})^2$ without taking sum_x^2 outside the sumcheck. π-Former proves a LayerNorm without divisions or square roots in five sub-protocols:

1. **Mean** (degree-2 sumcheck for $\widetilde{\text{sum\_x}}$).
2. **Square-sum** (cubic sumcheck for $\widetilde{\text{sq\_sum\_x}}(r_t) = \sum_j x_{\text{col}}(j)^3$ with three copies of x_{col}, keeping the round polynomial cubic).
3. **Sigma residual** (cubic multi-batched sumcheck combining $\sum_i \widetilde{\text{eq}}\,\widetilde{\text{sum\_x}}(i)^2$ and $\sum_i \widetilde{\text{eq}}(d\widetilde{\sigma}(i))^2$ under a Fiat–Shamir scalar λ_{sig}; the verifier reconstructs var_x algebraically).
4. **Floor-sqrt** (range-check $\text{var\_x} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var\_x} \geq 0$ via §3.5).
5. **Y fusion** (at one random (r_{y_t}, r_{y_d}), check $\gamma \odot (d\mathbf{x} - \text{sum\_x}) + \beta \cdot d\sigma \approx d\sigma \cdot \mathbf{y}$ via a range proof on the lo/hi residuals; the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs share one multi-batched cubic sumcheck).

Per LN the prover commits three internal Hyrax values $(\text{sum\_x}, \text{sq\_sum\_x}, \sigma)$; the squared quantities are bound algebraically by step 3 and need no separate commitment. The verifier evaluates γ, β from the VK in $O(d)$ per layer.

Model-level LN batching. After Phase 1 has absorbed all $5L+1$ LN IO commitments, one ProveLNsBatched call folds mean / variance / σ-floor / Y-fusion sub-claims of every LN into a single batched cubic sumcheck under Fiat–Shamir powers of an independent challenge. Soundness follows from the same Schwartz–Zippel argument as cross-block batching (Theorem 3) with L replaced by $5L+1$.

3.5 Globally batched range proof

The range proof shows $V[i] \in [0, 2^{\text{bits}})$ via chunked LogUp [6] over an identity table, with chunk width 16 and bucket widths $\{32, 64\}$. LayerNorm σ/y witnesses use the 64-bit bucket; each non-empty bucket produces one multiplicity commitment m_{com}.

For B witnesses in a bucket with $C = \lceil \text{bits}/16 \rceil$ chunks each, Phase 1 commits all $V_0^{(b)}, \ldots, V_{C-1}^{(b)}$ via Hyrax and merges chunk arrays into one global multiplicity vector $m[v] = \sum_{b,c,i} [\text{chunk}^{(b,c)}[i] = v]$ committed once. Phase 2 runs one degree-2 sumcheck per witness binding $V^{(b)}$ to a random $r_v^{(b)}$ and verifies $V^{(b)}(r_v^{(b)}) = \sum_c 2^{16c} V_c^{(b)}(r_v^{(b)})$ through one Hyrax batch open. Phase 3 opens $m(r_m)$ once and checks the LogUp identity against $T_{\text{id}}[i] = i$. The transcript-ordering invariant (Theorem 5) is that m_{com} is absorbed before any Phase 2 sumcheck challenge.

Zerocheck-folded inverse polynomial. The LogUp identity requires per-chunk inverse polynomials $h_c[i] = 1/(\alpha - V_c[i])$ for a Fiat–Shamir challenge α. A naïve implementation Hyrax-commits each h_c and opens it alongside the chunk commitments, costing $B \cdot C$ extra Pedersen MSMs per bucket. We avoid those commitments by folding a zerocheck of

$$h(x) \cdot (\alpha - V(x)) - 1 \;\equiv\; 0 \qquad (x \in \{0,1\}^n)$$

into the bucket's existing cubic sumcheck. Concretely, after the bucket claim $\Lambda = \sum_p w_p \cdot \big(\sum_i h_p[i] + \beta \cdot n_{\text{pad}}\big)$ is absorbed, the verifier samples $\gamma_{\text{fold}} \in \mathbb{F}$ and $r_{zc} \in \mathbb{F}^n$, and the prover proves

$$\sum_x \Big[w(x) \cdot h(x) \cdot \big(1 + \beta(\alpha - V(x))\big) + \gamma_{\text{fold}} \cdot \text{eq}(r_{zc}, x) \cdot \big(h(x)(\alpha - V(x)) - 1\big) \Big] =$$

The first term is the standard LogUp bucket claim; the second is the zerocheck term, which sums to zero on the boolean hypercube iff $h = 1/(\alpha - V)$ pointwise. The integrand remains cubic in (h, V, w, eq), so per-round prover cost is unchanged. We pre-fill h's pair-padding slots with α^{-1} so that the zerocheck term vanishes there ($\alpha^{-1} \cdot \alpha - 1 = 0$), keeping the bucket claim well-formed without an indicator mask.

Crucially, Λ is absorbed into the transcript *before* γ_{fold} and r_{zc} are sampled. The terminal $h(r_{\text{full}})$ is sent as a scalar (not opened against any commitment) and is bound only by the sumcheck's round-polynomial consistency together with the combined-identity check; the chunk evaluation $V(r_{\text{full}})$ remains bound to the Hyrax-committed V_c via the per-chunk openings at the bucket challenge r_k.

The model-level prover routes *all* range witnesses across the model into one bucketed list: $10L+2$ LN witnesses (σ, y for the five LNs per block, plus the final LN's two), and $2L$ further witnesses (rem, diff per block) when normalised attention is enabled. Sharing m_{com} per bucket saves $(B-1)\sqrt{2^{16}}$ G1 MSMs per bucket; the zerocheck fold removes a further $B \cdot C$ Hyrax commits and as many opens, with savings scaling with L rather than capped at the per-block level.

4 The π-Former Protocol

This section describes the setup, prover, and verifier at the level of the cross-block protocol structure. Detailed per-block and per-LN pseudocode is in Appendix **??**.

4.1 Setup

Algorithm 1 π-Former. $\mathsf{Setup}(\theta)$

Input: ternary weights and biases θ, lookup sub-tables $T_0, \ldots, T_{c-1}$, dimensions.
Output: $(\mathsf{pk}, \mathsf{vk})$.

1: derive transparent Hyrax generators g_j from SHA3 of public seeds
2: $C_{\mathbf{W}} \leftarrow \mathsf{Com}(\mathbf{W})$ for each weight matrix; $C_{T_i} \leftarrow \mathsf{Com}(T_i)$ for each sub-table
3: $\mathsf{vk} \leftarrow \{C_{\mathbf{W}}\} \cup \{C_{T_i}\} \cup \{\gamma, \beta\} \cup \{g_j\}$
4: $\mathsf{pk} \leftarrow \mathsf{vk} \cup \{\text{raw ternary arrays and sub-tables}\}$

The verifying key contains no raw weights, only commitments. The proving key additionally carries the ternary arrays so the prover can fold the equality polynomial against them in $O(Td)$ without field multiplications.

4.2 Model-level prover

Algorithm 2 π-Former. $\mathsf{Prove}(\mathsf{pk}, W, \mathsf{inst})$ outline

1: *Phase 1.* For each block ℓ: commit intermediates (per-block LN IO, ϕ inputs, attention auxiliaries); absorb in fixed order; chain residuals by Hyrax addition.
2: *Phase 2.* Draw one global random point $r_{td} \leftarrow$ FS.challenge shared by all cross-block sumchecks.
3: *Phase 3.* Run four cross-block sumchecks (QKV, output projection, attention out, attention context) via CROSSBATCH.
4: *Phase 4.* (When normalised attention is enabled) prove $S\mathbf{N} - \mathrm{rem} - Z\mathbf{Y}^{\text{att}} = 0$ and $Z - 1 - \mathrm{rem} - \mathrm{diff} = 0$ in one cubic multi-batched sumcheck; bind Z to $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$.
5: *Phase 5.* Commit FFN activations; prove the global $\phi(\mathbf{M})$ Lasso (committed-output) and two FFN cross-block sumchecks.
6: *Phase 6.* Run PROVERANGEBATCHED once per width bucket over all witnesses, and PROVELNSBATCHED once over all $5L+1$ LNs.
7: *Phase 7.* Prove the LM head; drain deferred-accumulator batching challenges and finalise in one MSM pair per Hyrax-parameter group; emit grouped batched opens at r_{td}, the per-protocol terminal points, and (when normalised) r_{norm}.
8: *Phase 8.* Prove the global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso, binding indices to the committed post-norm tensors.

Three structural choices drive the design. *(i)* A single global random point r_{td} is drawn after all per-block commitments are absorbed, so the six cross-block sumchecks share it. *(ii)* All final Hyrax openings flow through deferred batch accumulators (one per Hyrax-parameter group), each finalised in one MSM pair via random-linear-combination batching. *(iii)* Residual connections require no proof: by linearity of Hyrax (Theorem 4), $C_{\mathbf{X}_{\text{mid}}} = C_{\mathbf{X}_{\text{in}}} + C_{\text{LN}_{\text{aon}}.y}$ is computed by the verifier from the existing commitments.

Cross-block batched sumcheck. The primitive that powers Phases 3–5 reduces L per-block claims $\{H_\ell\}$ over per-block MLEs $\{f_\ell, g_\ell\}$ on $\mathbb{F}^n$ to a single sumcheck on $P(x) = \sum_\ell \eta^{\ell-1} f_\ell(x) g_\ell(x)$ at a Fiat–Shamir challenge η drawn after every per-block constant is absorbed.

Algorithm 3 CROSSBATCH$(\{f_\ell, g_\ell, H_\ell\}_\ell, \mathsf{FS})$

1: absorb $\{H_\ell\}$ and per-block constants; $\eta \leftarrow \mathsf{FS.challenge}$
2: $H^\star \leftarrow \sum_\ell \eta^{\ell-1} H_\ell$; run standard sumcheck on P to a final point $r \in \mathbb{F}^n$
3: verifier checks $P(r) = \sum_\ell \eta^{\ell-1} f_\ell(r) g_\ell(r)$ against per-block evaluations opened later in a batched open

By Theorem 3 this replaces L independent sumcheck tran-

scripts with one of length n at an additive $(L-1)/|\mathbb{F}|$ soundness cost.

Quantisation binding. Lasso queries are integer indices $\mathsf{idx}_j \in [0, 2^B)$, while committed input tensors carry signed field elements. Two quantisation sub-proofs bridge this gap. With public scales $(s_{\text{num}}, s_{\text{den}})$ and zero-point $\mathsf{zp} = 2^{B-1}$, the prover commits the remainder $\mathsf{rem}_j = r_j s_{\text{den}} + \lfloor s_{\text{num}}/2 \rfloor - s_{\text{num}}(\mathsf{idx}_j - \mathsf{zp})$, adds rem to the global range batch with bit-width $\log_2 s_{\text{num}}$, and at a Fiat–Shamir point $\mathbf{r}$ opens $\widetilde{r}$ and $\widetilde{\mathsf{rem}}$ via batched Hyrax. The verifier evaluates the public index MLE $\widetilde{\mathsf{idx}}(\mathbf{r})$ directly and checks

$$\widetilde{r}(\mathbf{r})\, s_{\text{den}} + \tfrac{s_{\text{num}}}{2}\widetilde{1}(\mathbf{r}) \stackrel{?}{=} s_{\text{num}}\big(\widetilde{\mathsf{idx}}(\mathbf{r}) - \mathsf{zp}\,\widetilde{1}(\mathbf{r})\big) + \widetilde{\mathsf{rem}}(\mathbf{r}). \quad (2)$$

Combined with the remainder range proof, this pins every public idx_j to the unique integer consistent with the committed raw value r_j.

Lasso instances. Both model-level Lasso instances are *committed-output* Lassos: the prover absorbs every Hyrax commitment to a lookup output before batching challenges are drawn, and an inner sumcheck binds the combined grand sum to the committed outputs at the sumcheck terminal. Lookup indices are bound to the committed post-norm input tensors via one shared Hyrax batch open, so a cheating prover can neither pick query indices independently of the inputs nor mix pre- and post-norm tensors.

4.3 Verifier

Algorithm 4 π-Former. $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y})$ outline

1: re-commit $\mathbf{x}_{\text{in}}$ and check against $\pi.C_{\mathbf{x}_{\text{in}}}$; initialise deferred Hyrax accumulators
2: mirror Phase 1 transcript absorption, computing residual commitments by Hyrax addition
3: draw r_{td}; verify the four cross-block sumchecks
4: (when normalised) verify the attention-normalisation sumcheck and the auxiliary Z sumcheck
5: verify the global FFN Lasso and the two FFN cross-block sumchecks
6: verify the bucketed range batch (one m_{com} per width)
7: verify the model-level batched LN proof over all $5L+1$ LNs
8: draw r_{lm}, evaluate $\widetilde{\mathbf{y}}(r_{\text{lm}})$ from the public output, verify the LM-head sumcheck; chain its input-side claim into the td accumulator against $\pi.C_{\text{final\_ln\_out}}$
9: drain deferred-accumulator batching challenges and finalise in one MSM pair per Hyrax-params group; verify all grouped batched openings
10: verify the global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso; return ACCEPT

5 Soundness

We state the main soundness theorems and outline the arguments here; full proofs appear in Appendix **??**.

Lemma 1 (Sumcheck soundness)**.** *A sumcheck for an MLE of individual degree $\leq \delta$ over $\{0,1\}^n$ has soundness error $\leq n\delta/|\mathbb{F}|$.*

Lemma 2 (Hyrax binding)**.** *Under discrete log on $\mathbb{G}$, the Hyrax PCS is computationally binding.*

Theorem 3 (Cross-block batching is sound)**.** CROSSBATCH *(Alg. 3) reduces L per-block claims to one batched claim $\sum_\ell \eta^{\ell-1} H_\ell$ with total soundness error $\leq (L-1)/|\mathbb{F}| + n\delta/|\mathbb{F}|$.*

Sketch. η is drawn after every per-block claim is absorbed, so the prover is committed to $\{H_\ell\}$ before seeing η. If any claim is false, the difference polynomial in η has degree $\leq L-1$ and is caught by Schwartz–Zippel with probability $\geq 1-(L-1)/|\mathbb{F}|$; the inner sumcheck adds $n\delta/|\mathbb{F}|$. □

Theorem 4 (Homomorphic residuals)**.** *For Hyrax commitments $C_{\mathbf{A}}, C_{\mathbf{B}}$, the verifier-computed commitment $C_{\mathbf{A}} + C_{\mathbf{B}}$ binds $\mathbf{A}+\mathbf{B}$.*

Sketch. By bilinearity of MSM over public generators, $C_{\mathbf{A}} + C_{\mathbf{B}} = \mathsf{Com}(\mathbf{A}+\mathbf{B})$; binding follows from Lemma 2. □

Theorem 5 (Globally batched range proof is sound)**.** *Sharing one m_{com} per width bucket across all range witnesses prevents any chunk value outside $[0, 2^{16})$. The zerocheck-folded inverse polynomial binds h to $1/(\alpha - V)$ on the boolean hypercube* without *committing h, with soundness error $O(n/|\mathbb{F}|)$ per bucket where n is the bucket variable count.*

Sketch. Two parts. *Chunk-value binding:* m_{com} is absorbed before any sumcheck challenge, so any chunk $v^* \geq 2^{16}$ has no matching T_{id} entry, breaking the LogUp identity. *Inverse-polynomial binding:* for malicious $\tilde{h} \neq 1/(\alpha - V)$, the zerocheck residual $\tilde{s}(x) = \tilde{h}(x)(\alpha - V(x)) - 1$ is non-zero on the hypercube, hence non-zero as a polynomial of degree $\leq 2n$; Schwartz–Zippel gives $\Pr_{r_{zc}}[\tilde{s}(r_{zc}) = 0] \leq 2n/|\mathbb{F}|$, the linear-in-$\gamma_{\text{fold}}$ check adds $1/|\mathbb{F}|$, and sumcheck adds $3n/|\mathbb{F}|$. Pair-padding $\tilde{h} = \alpha^{-1}$ keeps padding contributions zero in both terms. Full proof in Appendix **??**. □

Theorem 6 (Model-level batched LayerNorm is sound)**.** *The single* PROVELNSBATCHED *call covering all $5L+1$ LNs is as sound as $5L+1$ independent LN sub-proofs.*

Sketch. Apply Theorem 3 with L replaced by $5L+1$. □

Theorem 7 (Attention-normalisation sumcheck is sound)**.** *The cubic multi-batched sumcheck for $S\mathbf{N} - \text{rem} - Z\mathbf{Y}^{\text{att}} = 0$ and $\lambda(Z - 1 - \text{rem} - \text{diff}) = 0$ combined with range proofs on* rem, diff *and the auxiliary Z sumcheck soundly enforces $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S\mathbf{N}_{tj}/Z_t \rfloor$ pointwise.*

Table 1: π-Former Rust implementation, lines of code per module.

Module	LoC
`poly/` (dense MLE, equality, helpers)	∼0.6 k
`pcs/` (Hyrax PCS, batch accumulator)	∼1.3 k
`subprotocols/` (sumcheck ×3, GKR combine)	∼1.5 k
`lookup/` (Lasso multi, range batched, quant.)	∼3.4 k
`attention/` (LN, projection, LLA, ternary check)	∼4.6 k
`ffn/` (FFN sub-prover)	∼0.7 k
`cross_layer/` (cross-layer projection sumcheck)	∼0.6 k
`prover.rs`, `verifier.rs`, `setup.rs`	∼6.0 k
`bin/piformer/` (CLI, codecs, sample)	∼0.5 k
Total	∼19 k

Sketch. Range proofs give $0 \le \text{rem} < Z$ via the second identity; the first identity then expresses $Z\mathbf{Y}^{\text{att}}$ as the unique floor-division quotient. Both checks at one random point cost cubic sumcheck error plus $O(1)/|\mathbb{F}|$ batching error; the auxiliary Z sumcheck binds Z_t to the committed $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$. □

Theorem 8 (π-Former end-to-end soundness)**.** *For any PPT adversary $\mathcal{A}$,*

$$\Pr[\mathsf{Verify} = \mathsf{ACCEPT} \wedge \mathcal{M}_\theta(\mathbf{x}_{\text{in}}) \neq \mathbf{y}] \le \frac{m_{\text{sc}}\, n_{\max}\, \delta_{\max} + m_{\text{batch}}(L-1) + m_{\text{open}}}{|\mathbb{F}|} + \varepsilon_{\text{DL}}$$

where m_{sc} is the post-batching sumcheck count, $m_{\text{batch}} = 6$, m_{open} covers RLC checks, $n_{\max} \le 64$, $\delta_{\max} \le 3$, and ε_{DL} is $\mathcal{A}$'s DL advantage on $\mathbb{G}$. For typical parameters the statistical term is below 2^{-240}.

Sketch. Hybrid argument over the L blocks: any forged Hyrax open yields a DL solution (Lemma 2); residual chaining is sound by Theorem 4; per-block algebra is sound by Theorems 5, 6, 7, and 3; the committed-output Lasso is sound by Lemmas 2–1. A union bound over all sub-protocols yields the stated error. □

5.1 Zero knowledge

The current implementation is not zero-knowledge: weight commitments are public VK entries and intermediate sumcheck evaluations are revealed. Standard techniques apply — blinding witness polynomials before commitment, and zero-knowledge variants of sumcheck and Hyrax — which we leave to future work.

6 Implementation

We implement π-Former in ∼19 k lines of Rust over BN254 plus a PyTorch training pipeline. The Rust prover/verifier and CLI are organised as in Table 1.

PyTorch pipeline. The training pipeline implements `TernaryLinear` (with STE), `StructuredLookupActivation` (ϕ with sub-table forward plus STE backward), and `LinearAttentionLayer`. The integer forward pass used to construct the witness is bit-identical to the Rust circuit, so a witness exported from PyTorch is accepted by the Rust prover with no quantisation gap. The exporter also emits the Lasso instances ($\phi(\mathbf{Q}), \phi(\mathbf{K}), \phi(\mathbf{M})$ tables, query indices, and outputs) directly from the same integer arithmetic.

Single-head, causal supported, no ZK. The current Rust prover supports single-head attention with optional end-to-end causal masking (selected at setup; the prover swaps in three cubic-multi-batched sub-protocols whose third multiplicand absorbs the causal indicator polynomial) and no zero-knowledge masking layer. Multi-head splits the attention sumchecks into parallel batches with the same protocol structure, and ZK is an additional blinding layer over committed witnesses (§7).

7 Discussion and Limitations

What is and is not in the SNARK. The Rust prover proves the full normalised LLA output $\mathbf{Y}^{\text{att}} = \lfloor S \cdot \phi(\mathbf{Q}^{\text{n}})(\phi(\mathbf{K}^{\text{n}})^\top \mathbf{V})/Z \rfloor$ followed by the ternary output projection $\mathbf{O}_{\text{att}} = \mathbf{Y}^{\text{att}}\mathbf{W}_O + \mathbf{b}_O$ and the sandwich-norm $\text{LN}_{\text{aon}}(\mathbf{O}_{\text{att}})$ before the residual sum. The Python pipeline supports two attention modes: `normalized_fixed` (proven end-to-end by the Rust circuit, including the floor division) and `prover` (un-normalised numerator only, retained for legacy export); `normalized_float` is rejected at export time. *Causal masking is fully supported* end-to-end: when `causal=true` the prover swaps in cubic-multi-batched `batch_attn_out_causal` / `batch_attn_ctx_causal` / `attn_z_causal_sumcheck` sub-protocols that fold the causal indicator polynomial into the third multiplicand, so no separate prefix-context commitment is needed. The remaining limitations are multi-head attention (the prover currently rejects $n_{\text{heads}} \neq 1$ at setup) and zero-knowledge masking. Neither changes the protocol architecture: multi-head splits the attention sumchecks into parallel batches, and ZK is an additional blinding layer over committed witnesses.

Model-quality evaluation. Because π-Former changes the attention kernel and constrains weights to ternary values, a deployment-quality comparison must train matched baselines under the same parameter budget, context length, dataset, and quantisation regime, then report both task quality and proof cost. We leave this to future work and present π-Former here as evidence that the co-designed architecture is efficiently provable.

From prototype to deployment. Our reference implementation is CPU-only and single-threaded. GPU MSM and parallel sumcheck [12] would translate directly to π-Former: every dominant inner loop (cross-block batched sumcheck, Lasso multi-instance, batched Hyrax open) is data-parallel, and the global Hyrax accumulators are pure MSMs at the boundary. Streaming token generation is the natural fit for Nova-style folding [9]: the running context matrix $\mathbf{M} = \phi(\mathbf{K})^\top \mathbf{V}$ is the IVC accumulator, and one folding step per token yields constant amortised proving cost.

Beyond π-Former. The Rust codebase already contains two stand-alone sub-protocols that are not yet wired into the end-to-end prover. The first is an in-circuit ternary-weight check (`attention/ternary_check.rs`) via a 4-element Lasso table $[0, 1, r-1, 0]$; folding this into setup yields a verifying key that carries a binding proof $\mathbf{W} \in \mathcal{T}^{m\times n}$. The second is a single cubic sumcheck (`cross_layer/projection.rs`) that collapses L per-layer projection sumchecks into one of length $\log L + \log d_{\text{in}}$ by introducing a layer-index variable; this gives a further factor of L saving on the projection transcript. Together with an EVM-targeted Hyrax verifier and a zero-knowledge masking layer, these lay out the path from a research prototype to an on-chain verifiable inference service.

8 Related Work

Verifiable ML. zkCNN [10] gave the first GKR-based proof system for neural network inference, focused on convolutional networks. [1] extends ZK to proofs of *training* for deep networks. zkLLM [15] adapts a custom IOP to softmax LLM inference using `tlookup`-style structured lookups, with `zkAttn` as a tailored attention sub-protocol; zkGPT [12] uses a Plonky2 backend with constraint fusion across operations of a transformer block. Both target the standard softmax + GeLU + LayerNorm stack and pay a significant cost for every non-arithmetic operation. π-Former departs by co-designing the model and proof system: every operation is either a sumcheck or a Lasso lookup by construction, and the model is allowed to recover expressivity through learnable lookup tables that are free under Lasso.

Sumcheck and lookup arguments. The sumcheck protocol underlies modern transparent SNARKs [13, 16]. Lasso provides a practical lookup argument with sub-linear prover cost on structured-decomposable tables [2, 5, 14], and is the natural fit for the additive sub-table structure of π-Former's ϕ. LogUp-style multiplicity arguments [6] back the globally batched range proof. Hyrax [18] gives a transparent, discrete-log-based polynomial commitment with $O(\sqrt{N})$ proofs.

Linear attention and ternary networks. Linear attention [7] replaces softmax with a feature-map kernel and recovers the $O(Td^2)$-vs-$O(T^2d)$ complexity trade-off; [19] projects the keys and values to a low-rank space; Reformer [8] uses LSH for sparsity. BitNet b1.58 [11] introduced ternary weights as a replacement for FP16 dense layers in LLMs, motivated by inference energy on edge devices; π-Former re-purposes the same primitive for SNARK proving cost.

9 Conclusion

We presented π-Former, a SNARK for verifiable transformer inference in which the model is made SNARK-friendly so the proof system has nothing expensive to absorb. Three architectural changes drive the design: softmax becomes a kernel attention with a learnable additive-lookup feature map, every projection becomes ternary so its sumcheck contraction has no field multiplications, and LayerNorm becomes an integer reformulation provable through sumcheck and chunked range proofs. The proof system batches per-block sub-protocols into six model-level sumchecks, two global Lasso proofs, and one global range proof per block, with all final Hyrax openings flowing through ten deferred batch accumulators. We implemented the system in ∼19 k lines of Rust over BN254 and gave formal cross-block batching soundness theorems. We leave a deployment-quality model evaluation and the engineering steps (multi-head, ZK masking, GPU MSM, on-chain verifier) to future work; we believe π-Former provides a credible foundation for production-grade verifiable transformer inference.

References

[1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

[2] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Oleg Fomenko and Anton Levochko. Understanding lasso: A novel lookup argument protocol. *Cryptology ePrint Archive*, 2025.

[6] Ulrich Haböck. Improving logarithmic derivative lookups using gkr. *Cryptology ePrint Archive*, 2022.

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[8] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

[9] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[10] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[11] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. Bitnet b1. 58 2b4t technical report. *arXiv preprint arXiv:2504.12285*, 2025.

[12] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. {zkGPT}: An efficient non-interactive zero-knowledge proof framework for {LLM} inference. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2045–2063, 2025.

[13] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[14] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[15] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.

[16] Justin Thaler. *Proofs, arguments, and zero-knowledge*, volume 4. Foundations and Trends in Privacy and Security, 2022.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[18] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.

[19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.